RÉSOLUTION DES TRAVAUX PRATIQUES EC1

Coffre chiffré, infrastructure de gestion de clés, canal TLS et cycle de vie des certificats

Auteur : ZOUGOURY Donald

Module - Ingénierie de la cryptographie

Table des matières

1 TP 1 - Chiffrement authentifié et dérivation de clés **1**
1.1 Format CFR1 et dérivation 1
1.2 Implémentation et tests de sécurité 2
1.3 Questions de réflexion 2
1.4 Évolution des paramètres 3

2 TP 2 - Construire une infrastructure de gestion de clés **3**
2.1 Hiérarchie et profils 3
2.2 Rôle des extensions 3
2.3 Commandes et responsabilités 3
2.4 Émission, validation et révocation 4
2.5 Questions de réflexion 4
2.6 Politique minimale de certification 5

3 TP 3 - Canal TLS et authentification mutuelle **5**
3.1 Configuration Nginx 5
3.2 Poignée de main et vérification 6
3.3 Supervision et renouvellement 6
3.4 Questions de réflexion 6

4 Exécution, limites et remise **7**
4.1 Contrôles réalisés dans cet environnement 7
4.2 Règles de sécurité appliquées au dépôt 7

1 TP 1 - Chiffrement authentifié et dérivation de clés

1.1 Format CFR1 et dérivation

Le format est versionné par l'identifiant `CFR1`. Les trois paramètres Argon2id sont stockés dans l'en-tête afin que chaque fichier puisse être relu avec ses paramètres d'origine. Les entiers sont codés en big-endian sur quatre octets.

Champ	Offsets	Valeur et usage
Identifiant	0-3	`CFR1` ; sélection du format.
t, m, p	4-15	Trois entiers 32 bits : temps, mémoire en Kio, parallélisme.
Sel	16-31	16 octets aléatoires propres au fichier.
Nonce	32-43	12 octets aléatoires pour AES-GCM.
Chiffré + tag	44 et suite	Données chiffrées suivies du tag GCM de 16 octets.

Dérivation. Argon2id transforme le mot de passe et le sel en une clé maîtresse de 32 octets. Trois mesures médianes sur cet environnement donnent 0,220 s pour $t = 3$, $m = 65536$ Kio, $p = 4$, puis 0,595 s pour $t = 8$, $m = 131072$ Kio, $p = 4$. Ces derniers paramètres sont retenus pour viser environ 0,5 seconde ; le temps doit être re-mesuré sur la machine cible. `HKDF-SHA256` dérive la clé AES avec `info = coffre-v1|fichier`. Une étiquette de contexte sépare les usages.
Chiffrement. AES-GCM chiffre les données et authentifie le bloc de 44 octets avec le nom UTF-8 du fichier comme données associées. Le nom, l'en-tête, le nonce, le chiffré et son tag doivent tous rester cohérents pour que le déchiffrement réussisse.
Bornes de ressources. L'en-tête n'est pas authentifié avant la dérivation. Le parseur borne donc t, m et p avant d'appeler Argon2id, afin qu'un en-tête hostile ne puisse pas demander une allocation ou un temps sans limite. Toute évolution au-delà de ces plafonds exige une politique/version de format explicite.

1.2 Implémentation et tests de sécurité

`coffre.py` implémente `cle_maitresse`, `sous_cle`, `chiffrer`, `dechiffrer` et `rechiffrer`. Chaque appel de chiffrement tire un nouveau sel et un nouveau nonce via `os.urandom`. L'API AESGCM renvoie le chiffré avec son tag ; aucune primitive n'est réimplémentée.

Test	**Résultat attendu**	**Propriété**
Aller-retour	Le clair est retrouvé octet pour octet.	Correction
Deux chiffrements	Deux blobs différents.	Sel et nonce frais
Mauvais mot de passe	`InvalidTag`.	Authentification de la clé
Octet du chiffre modifié	`InvalidTag`.	Intégrité
Nom modifié	`InvalidTag`.	Liaison au contexte AAD
Rechiffrement	Nouveau mot de passe et paramètres lisibles.	Migration

1.3 Questions de réflexion

1. **Pourquoi le sel est-il public ?** Le sel n'est pas un secret. Il rend la clé dérivée propre à ce fichier et empêche de réutiliser des tables précalculées ou de détecter directement deux mêmes mots de passe. Le publier n'affaiblit pas Argon2id ; la protection vient du mot de passe, du coût de dérivation et de la taille du sel.
2. **Pourquoi le nonce aléatoire ne se répète-t-il pas sous une même clé ?** Chaque fichier obtient un sel différent et donc, sauf collision de sel, une clé AES différente. La règle GCM est l'unicité du nonce par clé, pas l'unicité globale entre toutes les clés. Avec un million de sels uniformes sur 128 bits, la probabilité de collision est de l'ordre de 10^{-27}. Pour un même fichier, une nouvelle encryption produit aussi un nouveau sel et nonce.
3. **Que risque-t-on sans le nom en données associées ?** Un attaquant pourrait déplacer ou renommer un blob valide vers un autre emplacement. L'application pourrait alors traiter son clair dans le mauvais contexte (par exemple, charger un fichier de configuration sous un nom réservé aux autorisations) sans que l'authentification détecte le changement.
4. **Pourquoi authentifier l'en-tête ?** Sans l'en-tête dans AAD, un attaquant pourrait modifier la version ou les paramètres Argon2id sans que le tag lie ces choix au contenu. Il pourrait rendre le fichier illisible, exploiter des paramètres faibles acceptés ou provoquer un coût de calcul excessif. Les bornes de ressources restent nécessaires car elles sont vérifiées avant le tag.
5. **Pourquoi une erreur générique ?** Une différence visible entre mauvais mot de passe, tag invalide, format corrompu ou paramètres invalides pourrait servir d'oracle et donner des indices à un attaquant. L'interface doit renvoyer un seul message de rejet ; les détails techniques peuvent rester dans des journaux internes protégés.

1.4 Évolution des paramètres

L'en-tête conserve les paramètres ayant servi au fichier ; déchiffrer lit donc les anciens paramètres, tandis qu'un nouveau chiffrement utilise les valeurs courantes. `rechiffrer` ouvre le blob avec l'ancien mot de passe, puis crée un nouveau blob avec le nouveau mot de passe, un nouveau sel, un nouveau nonce et des paramètres plus forts. Une évolution structurelle ou un changement d'algorithme utiliserait un nouvel identifiant, par exemple `CFR2`, sans supprimer le lecteur `CFR1`.
Résultat des tests. Six tests automatisés passent : les cinq propriétés demandées et le rechiffrement avec renforcement des paramètres. L'implémentation propage `InvalidTag` au niveau bas ; l'application appelante doit uniformiser le message présenté à l'utilisateur.

2 TP 2 - Construire une infrastructure de gestion de clés

2.1 Hiérarchie et profils

Certificat	Clé/durée	Extensions et contrainte
Racine	EC P-384/10 ans	`CA:TRUE`, `pathlen:1` ; `keyCertSign`, `cRLSign` ; auto-signée et hors ligne.
Intermédiaire	EC P-256/5 ans	`CA:TRUE`, `pathlen:0` ; contraintes DNS et courriel limitées à `tp.local`.
Serveur `www.tp.local`	EC P-256/90 jours	`CA:FALSE` ; `digitalSignature` ; `serverAuth` ; SAN DNS `www.tp.local` et `tp.local`.
Client Alice/Bob	EC P-256/90 jours	`CA:FALSE` ; `digitalSignature` ; `clientAuth` ; SAN `alice@tp.local` ou `bob@tp.local`.

2.2 Rôle des extensions

`basicConstraints` distingue une AC des certificats terminaux et borne la profondeur de la chaîne. `pathlen:0` interdit à l'intermédiaire d'émettre une sous-AC. `keyUsage` limite l'usage de la clé ; seules les AC ont `keyCertSign` et `cRLSign`. `extendedKeyUsage` sépare les certificats serveur (`serverAuth`) des clients (`clientAuth`). `subjectAltName` porte les noms vérifiés par les clients. `subjectKeyIdentifier` et `authorityKeyIdentifier` facilitent la construction de la chaîne. Les extensions d'AC qui fixent la confiance sont critiques.

L'intermédiaire déclare des contraintes de nom critiques pour DNS `tp.local` et le domaine de courriel `tp.local`. Les noms permis incluent `tp.local` et ses sous-domaines pour DNS ; les identités de client restent dans le domaine de laboratoire.

2.3 Commandes et responsabilités

```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out racine
    .key
openssl req -x509 -new -key racine.key -sha384 -days 3650 -addext "
    basicConstraints=critical,CA:TRUE,pathlen:1" -addext "keyUsage=critical
    ,keyCertSign,cRLSign" -out racine.crt
openssl ca -batch -config racine.cnf -extensions ext_inter -days 1825 -in
    inter/inter.csr -out inter/inter.crt
```

`pki/pki.sh` reprend ces étapes avec des fichiers de configuration, une base CA, des numéros de série, des clés d'autorité chiffrées et une saisie silencieuse des phrases secrètes. Les clés privées et les CSR sont ignorées par Git. Dans une vraie exploitation, la racine est générée et utilisée lors d'une cérémonie hors ligne ; le script pédagogique ne remplace pas cette séparation physique.

2.4 Émission, validation et révocation

Le demandeur génère sa clé P-256 et une CSR signée par cette clé. L'AC valide l'identité et choisit elle-même l'extension autorisée ; la configuration utilise `copy_extensions=none` pour ne pas laisser le demandeur injecter `basicConstraints` ou d'autres extensions sensibles. La clé privée ne quitte pas le serveur ou le client.

```
openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt -
   verify_hostname www.tp.local -purpose sslserver inter/serveur.crt
openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt -
   purpose sslclient inter/alice.crt
```

Le premier contrôle doit réussir ; le même certificat avec `autre.example` doit échouer au contrôle de nom. Le certificat client doit être accepté pour `sslclient`. Le certificat de démonstration `www.example.com` est signé par l'intermédiaire, mais la validation de la chaîne doit échouer avec le message `permitted subtree violation`. L'émission n'applique pas nécessairement les contraintes de la chaîne ; le validateur, lui, les applique.

Alice est révoquée avec la raison `keyCompromise`. L'intermédiaire publie une CRL ; la racine publie aussi la sienne. Les deux CRL sont concaténées dans `crls.pem`, et `openssl verify -crl_check_all` doit marquer Alice comme révoquée. Bob reçoit ensuite un certificat neuf ; le certificat d'Alice ne doit plus être accepté.

2.5 Questions de réflexion

1. **Compromission.** La racine reste hors ligne pour réduire l'exposition. Si l'intermédiaire est compromis, on suspend son émission, on le révoque avec la racine, on publie la nouvelle CRL, on crée un intermédiaire de remplacement et on réémet les feuilles. Si la racine est compromise, la chaîne de confiance est perdue : il faut créer une nouvelle racine, la distribuer hors bande, retirer l'ancienne des magasins de confiance et réémettre toute la hiérarchie.
2. `pathlen:0`**.** Il autorise l'intermédiaire à signer des feuilles mais pas d'autres AC. Sans cette limite, une AC compromise pourrait déléguer l'émission et élargir le nombre d'acteurs capables de créer des certificats.
3. **Clé du serveur.** Le serveur crée sa clé privée et envoie seulement une CSR. La clé ne traverse pas le réseau et l'AC ne la connaît jamais ; cela réduit les copies et les risques de fuite.
4. **CN et SAN.** Les clients modernes comparent le nom d'hôte avec le SAN. Le CN est une information de sujet et ne remplace pas un SAN correct.
5. **CRL, OCSP, durée courte.** Une CRL peut être volumineuse et son contenu devient *stale* entre deux publications. OCSP donne un état plus récent, mais ajoute latence, dépendance au répondeur et fuite d'information sur les sites visités ; certains clients ont aussi pratiqué l'échec ouvert. Des certificats courts réduisent la fenêtre d'exposition, au prix d'un renouvellement automatisé et surveillé.

2.6 Politique minimale de certification

Règle	Politique proposée
1. Éligibilité	Seuls les administrateurs et services inscrits à l'inventaire peuvent demander un certificat.
2. Identité	Un opérateur autorisé valide le compte ; un contrôle DNS ou une approbation de propriétaire confirme les noms.
3. Clé	Le demandeur génère une clé EC P-256 localement ; aucune clé privée n'est envoyée à l'AC.
4. CSR	La CSR doit prouver la possession de la clé et correspondre au sujet demandé.
5. Émission	L'intermédiaire émet seulement les profils serveur ou client approuvés et journalise série, sujet, SAN et opérateur.
6. Durée	Feuilles valides 90 jours ; renouvellement automatisé avant 30 jours restants.
7. Protection	La racine chiffrée est conservée hors ligne ; l'intermédiaire est protégée par contrôle d'accès et phrase secrète hors dépôt.
8. Révocation	Une compromission est signalée immédiatement, le certificat est révoqué et une CRL est publiée.
9. Distribution	Les clients font confiance uniquement à la racine de laboratoire ; les serveurs envoient feuille et intermédiaire, jamais la racine.
10. Audit	Les émissions, renouvellements, échecs et révocations sont journalisés, l'inventaire et les dates sont contrôlés quotidiennement.

Pourquoi l'émetteur et le sujet de la racine sont identiques ? Le certificat racine est autosigné. Cette signature protège le contenu du certificat mais ne crée pas la confiance : celle-ci provient de l'installation explicite de la racine dans le magasin de confiance.

3 TP 3 - Canal TLS et authentification mutuelle

3.1 Configuration Nginx

Directive	Effet et justification
`ssl_certificate / ssl_certificate_key`	Le serveur présente feuille + intermédiaire et utilise sa clé privée locale.
`ssl_protocols TLSv1.2 TLSv1.3`	Refuse SSL et TLS 1.0/1.1.
`ssl_ciphers`	Pour TLS 1.2, limite le choix à ECDHE-ECDSA avec AES-GCM ou ChaCha20-Poly1305, donc PFS et AEAD.
`ssl_session_tickets off`	Désactive les tickets *stateless* tant que les clés de ticket ne sont pas gérées avec rotation.
`ssl_client_certificate / ssl_verify_client on`	Demande un certificat client et valide sa chaîne vers la racine.
`ssl_verify_depth 2 / ssl_crl`	Borne la profondeur et vérifie les CRL concaténées.
`Strict-Transport-Security`	Demande au navigateur d'utiliser HTTPS pendant un an après réception de l'en-tête.
`$ssl_client_s_dn`	Affiche l'identité X.509 du client authentifié pour le service de laboratoire.

Les suites TLS 1.3 sont gérées par OpenSSL et ne sont pas régies par `ssl_ciphers` dans les versions courantes de Nginx. Elles doivent rester sur des suites AEAD modernes. La configuration limite ici les suites TLS 1.2 compatibles avec le certificat ECDSA.

3.2 Poignée de main et vérification

Le client envoie `ClientHello` avec les versions, suites, aléa et `key_share`. Le serveur répond `ServerHello` avec la suite et son `key_share`. Les deux parties dérivent les clés éphémères ; le serveur envoie ensuite, chiffré, la chaîne, `CertificateVerify` et `Finished`. `CertificateVerify` signe la transcription ; `Finished` confirme les secrets dérivés.

En TLS 1.3, le certificat serveur n'est pas visible en clair sur le réseau : après `ServerHello`, les messages suivants de la poignée de main sont chiffrés sous les clés dérivées. Wireshark peut les afficher si on lui fournit les secrets de session dans un fichier de *key log*. En TLS 1.2, le certificat est normalement visible.

```
openssl s_client -connect www.tp.local:443 -servername www.tp.local -
    verify_hostname www.tp.local -CAfile racine/racine.crt -cert bob.crt -
    key bob.key -cert_chain inter/inter.crt -trace
```

Sur la machine de labo, relever dans la sortie le protocole négocié, la suite et `Verification: OK`. Le groupe d'échange se relève dans les extensions `key_share` avec `-trace`. Sans certificat client, le serveur doit refuser la connexion ; avec Alice, la validation doit échouer après sa révocation. TLS 1.1 doit échouer.

3.3 Supervision et renouvellement

`tls/surveiller.py` parcourt récursivement les PEM/DER. Pour chaque certificat, il compare l'expiration à l'heure UTC : OK à partir de 30 jours, ALERTE entre 7 et 30 jours, CRITIQUE sous 7 jours ou après expiration. Le code de sortie est le risque maximal : 0, 1 ou 2. Un certificat illisible ou un inventaire vide produit 2 pour éviter un faux état sain.

Le test `tls/test_surveiller.py` fabrique temporairement un certificat valide cinq jours avec une clé gardée en mémoire. Le script de surveillance le classe CRITIQUE et renvoie 2 ; aucun secret n'est écrit dans le dépôt.

`tls/renouveler.sh` quitte sans changement si le certificat a encore plus de 30 jours. Sinon, il génère une nouvelle clé et CSR sur le serveur, fait signer la CSR, valide la chaîne et la correspondance clé/certificat, sauvegarde les anciens fichiers, installe les nouveaux, exécute `nginx -t` puis recharge Nginx. Une erreur pendant le déploiement restaure les fichiers précédents et tente un nouveau contrôle/rechargement.

3.4 Questions de réflexion

1. **Pourquoi ne pas envoyer la racine ?** Le client doit déjà faire confiance à la racine par son magasin local. La transmettre n'ajoute pas cette confiance ; le serveur envoie le certificat feuille et les intermédiaires nécessaires à la construction du chemin.
2. **Tickets de session.** Les tickets sont chiffrés avec une clé de serveur. Si cette clé n'est pas renouvelée, un attaquant qui la récupère peut conserver une capacité de déchiffrement ou de reprise plus longtemps. Les désactiver évite ce risque de clé longue durée ; si on les active, la clé de ticket doit être gérée et tournée.
3. **HSTS.** Après réception, le navigateur remplace les accès HTTP par HTTPS pendant `max-age` et refuse les contournements de certificat. Cela protège contre le *downgrade* et le *SSL stripping* pour les visites suivantes ; le premier contact n'est protégé que si le domaine est préchargé. Cette configuration ne couvre pas les sous-domaines car `includeSubDomains` n'est pas présent.
4. **Durées publiques.** Selon le calendrier donné dans le cours, la limite passe à 100 jours en 2027 puis 47 jours en 2029. Une organisation qui renouvelle à la main augmente fortement le risque d'oubli et d'indisponibilité ; inventaire, alertes, renouvellement automatisé et retour arrière deviennent nécessaires.

5. **Chaîne incomplète.** Certains clients possèdent déjà l'intermédiaire en cache ou savent le récupérer via les informations d'accès à l'autorité ; d'autres n'ont ni cache ni accès et échouent. Le serveur doit envoyer la feuille et les intermédiaires, pas dépendre de l'état du client.
6. **Fuite de clé serveur.** Isoler le serveur et informer l'AC ; révoquer le certificat ; générer une nouvelle paire sur une machine saine ; émettre et vérifier un nouveau certificat ; déployer la chaîne sans interruption ; publier/rafraîchir les CRL ; invalider les sessions et tickets concernés ; rechercher les traces d'exploitation et informer les parties concernées.

4 Exécution, limites et remise

4.1 Contrôles réalisés dans cet environnement

Élément	**État**
Tests du coffre	6 tests pytest passent : aller-retour, aléa, mauvais secret, modification, renommage et migration.
Surveillance 5 jours	Test exécuté : le certificat est classé CRITIQUE et le code de sortie vaut 2.
Syntaxe Python	Les modules et scripts Python compilent.
Syntaxe Bash	`pki.sh` et `renouveler.sh` passent le contrôle `bash -n`.
PKI avec OpenSSL 3.2.4	`pki.sh` exécuté sur copie temporaire : racine/intermédiaire, SAN serveur, contraintes, CRL ; `www.example.com` est rejeté et Alice révoquée tandis que Bob est validé.
Nginx/handshake TLS	Nginx n'est pas installé ici ; la configuration, mTLS, TLS 1.1 et la suite/groupe négociés restent à relever sur le serveur Linux.

La chaîne PKI, les usages, le mauvais nom, la violation de `nameConstraints` et la CRL ont été vérifiés avec OpenSSL 3.2.4 dans un répertoire temporaire ensuite nettoyé. Les résultats de suite TLS, de groupe ECDHE, de vérification Nginx et les captures restent dépendants de l'exécution du serveur Linux. Il faut y reporter la sortie réelle avant de présenter une capture ou une mesure comme observation.

4.2 Règles de sécurité appliquées au dépôt

Le dépôt contient le code, les configurations, le rapport et des scripts de génération ; il ne contient aucun mot de passe, aucune clé privée, aucune CSR ni base d'émission. Les clés de laboratoire générées localement sont ignorées par Git. Les secrets de renouvellement doivent venir d'un stockage protégé, jamais d'une ligne de commande contenant la phrase secrète ni d'un fichier versionné.